

Pre-final Implementation Status Report

RISC-V Simulator Pipelining

Rishit Mittal
Roll No: CS24BTECH11053

November 17, 2025

Project Status Report

Abstract

This report details the successful implementation of a multi-mode, 5-stage pipelined processor, evolving from the previous single-cycle design. The simulator now supports six distinct execution modes (0-5), correctly handling data hazards (via stalling and forwarding) and control hazards (via static and dynamic branch prediction). The core logic has been heavily refactored for simulation accuracy, notably by implementing a "double-buffered" pipeline register system to prevent race conditions. All integer and branch instructions are now correctly processed. The next steps are to add support for complex instructions (M/F/D extensions) and R4-type instructions.

1 Implementation Architecture

The pipeline's core logic was completely refactored to support multiple modes and ensure correct simulation of parallel hardware.

1.1 Double-Buffered Pipeline Registers

A critical simulation flaw was identified where sequential C++ execution of pipeline stages (e.g., `pipeline_mem` running before `pipeline_execute`) caused a data race condition. This was solved by implementing a "double-buffer" system.

- I duplicated all pipeline registers into a "current" set (e.g., `id_ex_reg`) and a "next" set (e.g., `id_ex_reg_next`).
- All pipeline stages now **read from** the "current" registers and **write to** the "next" registers.
- At the end of `Clocktick()`, a "latch" phase copies all "next" registers to the "current" registers, perfectly simulating a single clock edge and eliminating the race condition.

1.2 Multi-Mode Execution

The `Run()` and `Step()` functions now act as routers. They read the `pipeline_mode` from the config file and call either `Run_single()` (Mode 0) or `Run_Pipelined()` (Modes 1-5). This cleanly separates the single-cycle and pipelined logic paths.

1.3 Modular Control Units

All complex hazard and control logic has been removed from pipeline stages and modularized into three independent units:

- **HazardDetectionUnit (HDU):** The main brain for data hazards.
- **BranchComparator:** Evaluates branch conditions in Decode for Mode 4.
- **BranchPredictionUnit (BPU):** State-based prediction (2-bit counters + BTB) for Mode 5.

2 Mode Verification & Results

I have successfully implemented and verified all 6 modes using custom tests.

- **Mode 0 (Single-Cycle):** Correct.
- **Mode 1 (Naive Pipeline):** Incorrect results, proving hazards exist.
- **Mode 2 (Stall-Only):** Correct results, many stalls.

- **Mode 3 (Forwarding):** Correct results, faster, forwards correctly.
- **Mode 4 (Static Prediction):** 1-cycle penalty on mispredict.
- **Mode 5 (Dynamic Prediction):** Reduced stalls from 9 to 2 in loop.

3 Challenges Addressed

- **Simulation Race Condition:** Fixed using double-buffered registers.
- **HDU Forwarding Logic Bug:** Fixed by checking EX/MEM/WB order.
- **Incomplete Flush Loop Bug:** Fixed by dynamic valid-bit loop.

4 Next Steps & Future Enhancements

1. **Support Complex Instructions (F, D, M, R4):**
 - **R4-Type:** Support `rs3` for `fmadd.s`.
 - **FPRs:** Add float register file and support in all stages.
 - **HDU Upgrade:** Handle new FPR-related dependencies.
2. **Implement Cache Hierarchy:** Add basic L1/L2 + stall signals.
3. **Create Trace Visualizer:** Dump `trace.json` for visualization.

5 Additional Advancements / Improvements

- **Graphical User Interface (GUI):** Add interactive visual pipeline tracking, single-stepping, hazard highlighting, and register charting using Qt or ImGui.
- **Advanced Branch Predictors:** Add GShare, tournament, hybrid predictors, global history buffers, and perceptron-based prediction.
- **Extended Pseudo Instructions:** Add `li`, `sxt.w`, `seqz`, `snez`, `push`, `pop`.
- **Beginner-Level Cache Model:** Implement direct-mapped or 2-way set associative cache with adjustable hit/miss latency.
- **Performance Counters:** Track IPC, stall cycles, memory wait cycles, mispredict rate, and execution time reports.
- **Instruction Visualization Tools:** Generate cycle-by-cycle animation logs for external visualization engines.